GUÍA PRÁCTICA DE BASES DE DATOS RELACIONALES

Sistema de compras a proveedores

Creación, inserción de datos y consultas en PostgreSQL

- Validación de datos con restricciones CHECK, UNIQUE y DEFAULT
- Integridad referencial con claves foráneas y reglas ON DELETE / ON UPDATE
- Normalización hasta la Tercera Forma Normal (3FN)
- Triggers, columnas calculadas y vistas

Contenido del ejercicio

12	tablas
156	ciudades en 20 departamentos
20	proveedores y 20 productos
60	facturas de compra y 30 órdenes
15	pruebas de validación
17	consultas guiadas

Motor: PostgreSQL 13 o superior · Todos los scripts y resultados fueron ejecutados y verificados en PostgreSQL 16

Contenido

1. Presentación 3

Objetivos de aprendizaje 3

Cómo usar esta guía 3

2. Caso de estudio 3

3. Diagnóstico del script original 4

4. Normalización paso a paso 6

Primera Forma Normal (1FN): valores atómicos 6

Segunda Forma Normal (2FN): dependencia de toda la clave 6

Tercera Forma Normal (3FN): sin dependencias transitivas 6

5. Modelo relacional resultante 8

Resumen de tablas e integridad referencial 9

6. Creación de la base de datos (DDL) 10

6.1 Limpieza y parámetros 10

6.2 Geografía: departamentos y ciudades 10

6.3 Proveedores 11

6.4 Productos y catálogo por proveedor 11

6.5 Áreas y órdenes de compra (encabezado–detalle) 12

6.6 Facturas y pagos 12

7. Inserción de datos 14

Buenas prácticas aplicadas 14

8. Pruebas de validación e integridad 16

9. Consultas 19

Nivel 1 · Consultas básicas 19

Nivel 2 · Combinación de tablas y agregación 20

Nivel 3 · Consultas avanzadas 23

10. Ejercicios propuestos 26

Pistas de autoevaluación 26

1. Presentación

Esta guía convierte el script *script_bd_prueba.sql* en un ejercicio de aprendizaje completo. El script original crea una base de datos de compras a proveedores y la llena con datos de ejemplo; funciona, pero deja pasar errores que una base de datos bien diseñada debería impedir: valores negativos, datos repetidos, totales que no cuadran o pagos que no corresponden a la factura. El objetivo es analizar esas debilidades, corregirlas con las herramientas del propio motor y comprobar, con pruebas reales, que la nueva estructura protege la información.

Objetivos de aprendizaje

Al terminar el ejercicio, el estudiante podrá identificar violaciones a la Primera, Segunda y Tercera Forma Normal en un esquema existente y proponer su corrección; declarar validaciones de dominio con **CHECK**, **UNIQUE**, **NOT NULL** y **DEFAULT**; definir claves foráneas eligiendo conscientemente entre **RESTRICT** y **CASCADE**; implementar reglas de negocio que cruzan varias tablas mediante **triggers**; evitar datos derivados inconsistentes con **columnas calculadas** y **vistas**; cargar datos respetando el orden de dependencias dentro de una transacción; y escribir consultas de complejidad creciente, desde un SELECT simple hasta funciones de ventana y expresiones de tabla comunes (CTE).

Cómo usar esta guía

Se recomienda seguir las secciones en orden. Las secciones 3 y 4 son de análisis: lea los hallazgos e intente explicar por qué cada uno es un problema antes de leer la corrección. Las secciones 6 y 7 contienen el script que se debe ejecutar. La sección 8 propone «romper» la base de datos a propósito: ejecute cada instrucción y compare el error obtenido con el esperado. La sección 9 presenta 17 consultas con su resultado real, y la sección 10 propone ejercicios para practicar. El Anexo A reúne el script completo, listo para copiar.

> **Requisitos.** PostgreSQL 13 o superior (las columnas `GENERATED ALWAYS AS ... STORED` existen desde la versión 12 y `EXECUTE FUNCTION` en triggers desde la 11) y un cliente como pgAdmin, DBeaver o psql. Para ejecutar el script desde psql: `\i script_compras_normalizado.sql`

2. Caso de estudio

Una empresa compra tecnología y mobiliario a proveedores de distintas ciudades de Colombia. Cada **área** interna (Compras, Ventas, Tecnología, Operaciones...) genera **órdenes de compra** con los productos que necesita. Los **proveedores** tienen un catálogo con el precio al que venden cada **producto**; un mismo producto puede ser ofrecido por varios proveedores a precios diferentes. Cuando se realiza una compra, el proveedor emite una **factura** compuesta por un encabezado (fecha, proveedor, ciudad de entrega) y un detalle (productos, cantidades, precios, descuentos e IVA). Las facturas se pagan en una o varias **cuotas**, y la empresa necesita conocer en todo momento el saldo pendiente de cada una. Una tabla de **parámetros** guarda los días de la semana en que se hacen pagos y el consecutivo de la próxima orden.

Las reglas de negocio que la base de datos debe garantizar son: toda ciudad pertenece a un departamento existente; un proveedor solo puede facturar productos de su catálogo; los valores monetarios y cantidades son positivos; el IVA es uno de los porcentajes vigentes en Colombia (0 %, 5 % o 19 %); el valor neto de una línea es siempre bruto − descuento + IVA; ningún pago puede ser anterior a la factura ni superar su saldo; y no se puede borrar información que tenga registros dependientes importantes, como un departamento con ciudades o una factura con pagos.

3. Diagnóstico del script original

Antes de corregir hay que medir. El script original se ejecutó tal cual en PostgreSQL 16 y luego se auditó con consultas. Se ejecuta sin errores, lo que confirma algo importante: **que un script corra no significa que los datos sean correctos**. La tabla resume los hallazgos, el principio de diseño afectado y la corrección aplicada en la versión normalizada.

N.º	Hallazgo en el script original	Principio	Corrección aplicada
1	`tab_prov.datos_prov` usa el tipo compuesto `datos` (dirección, teléfono y correo en un solo campo).	1FN	Tres columnas atómicas: dir_prov, tel_prov, email_prov, cada una con su validación.
2	`tab_orden_compra` admite un solo producto por orden; pedir dos productos exigiría repetir id_orden, lo que la clave primaria impide.	1FN	Separación en encabezado (tab_orden_compra) y detalle (tab_det_orden).
3	`nom_comp` repite nombre y apellido ya almacenados en otras columnas.	3FN	Apellidos separados (ape1_prov, ape2_prov) y nom_comp como columna calculada.
4	`val_bruto, val_iva, val_neto` se escriben a mano: en 4 líneas bruto − descuento + IVA ≠ neto, y el IVA aplicado oscila entre 16,05 % y 18 % (el vigente es 19 %).	3FN / validación	val_unit y porc_iva como datos base; bruto, IVA y neto calculados por el motor.
5	`tab_pagos.val_saldo` es un dato derivado. En 4 de las 10 facturas con pagos (200001, 200002, 200004 y 200008) la suma pagada no coincide con el total y aun así el saldo final registrado es 0.	3FN / validación	Se elimina val_saldo; el saldo se calcula en la vista vw_estado_factura y un trigger impide pagar de más.
6	`tab_enc_compra.id_ciud` es ambiguo: en 48 de 60 facturas no coincide con la ciudad del proveedor. Si fuera la ciudad del proveedor sería una dependencia transitiva.	3FN	Se interpreta como ciudad de entrega y se renombra id_ciud_entrega.
7	No hay ninguna restricción CHECK: cantidades, precios, cupos y puntajes aceptan negativos o valores fuera de rango.	Validación	CHECK en todas las columnas numéricas, de formato y de rango.
8	`tab_det_compra` solo referencia a tab_prod: se podría facturar un producto que el proveedor no vende.	Integridad	Trigger trg_valida_det_compra contra el catálogo tab_prod_prov.
9	Claves foráneas sin nombre y sin acción explícita (NO ACTION por defecto).	Integridad	Restricciones nombradas (pk_, fk_, ck_, uq_) con RESTRICT o CASCADE según el significado.
10	`fec_orden` es DATE y se insertan valores con hora: la hora se pierde sin aviso. Además admite NULL.	Validación	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP.
11	Teléfono como DECIMAL(10,0): perdería ceros a la izquierda y permite sumarlo. Dos proveedores tienen indicativo 603, que no existe.	Validación	VARCHAR(10) con patrón de indicativos válidos; 603 corregido a 606.
12	Puerto Nare y Puerto Triunfo (Antioquia), Puerto López y Puerto Concordia (Meta) aparecen en Guaviare; Puerto López queda duplicado.	Calidad de datos	Registros eliminados; Guaviare conserva sus 4 municipios. UNIQUE (id_dpto, nom_ciud).
13	Todos los proveedores están en Medellín, aunque sus indicativos telefónicos corresponden a otras regiones.	Calidad de datos	Ciudades reasignadas de forma coherente con el indicativo.
14	`num_orden` vale 100001, pero las órdenes 100001 a 100030 ya existen: la próxima orden chocaría con la clave primaria.	Integridad	Consecutivo actualizado a 100031.
15	34 de las 60 facturas no tienen ninguna línea de detalle.	Calidad de datos	Se detectan con una consulta (sección 9) y se dejan como ejercicio.

N.º	Hallazgo en el script original	Principio	Corrección aplicada
16	VARCHAR sin longitud y DECIMAL(1,0) para días de pago.	Validación	Longitudes explícitas; días como SMALLINT entre 1 y 7 en orden creciente.

Los códigos de departamento (01 a 20) y de ciudad son secuenciales y no corresponden a la codificación oficial DIVIPOLA del DANE (por ejemplo, Antioquia es 05 y Bogotá D.C. es 11001). Para el ejercicio se conservan, pero en un sistema real se deben usar los códigos oficiales.

4. Normalización paso a paso

Normalizar es organizar las columnas en tablas de forma que cada dato se guarde **una sola vez** y en el lugar que le corresponde. Así se evitan tres anomalías: de *inserción* (no poder registrar un dato sin inventar otro), de *actualización* (cambiar un dato en un sitio y olvidar sus copias) y de *borrado* (perder información al eliminar una fila). Se avanza por formas normales; cada una exige cumplir la anterior.

Primera Forma Normal (1FN): valores atómicos

Una tabla está en 1FN si cada celda contiene un único valor indivisible y no hay grupos repetitivos. El tipo compuesto `datos` incumple la regla: guarda tres datos en una celda. Consultar los correos obliga a escribir `(datos_prov).correo`, no se puede declarar de forma natural que el correo sea único, y validar su formato resulta incómodo.

```
-- ANTES: tres datos en una sola columna
datos_prov  datos   -- ROW('Calle 100 #45-20', 6011234567, 'carlos@tecnosum.com')

-- DESPUÉS: una columna por dato, cada una con su propia regla
dir_prov    VARCHAR(100) NOT NULL,
tel_prov    VARCHAR(10)  NOT NULL,   -- CHECK con patrón de indicativos
email_prov  VARCHAR(100) NOT NULL,   -- CHECK de formato + UNIQUE
```

La orden de compra presenta el segundo problema de 1FN: un grupo repetitivo «escondido». Como la orden tiene un solo par (producto, cantidad), una orden con tres productos tendría que registrarse como tres órdenes distintas. La solución clásica es el patrón **encabezado-detalle**: los datos que ocurren una vez por orden (fecha, área) van al encabezado y los que se repiten (producto, cantidad) van al detalle.

Segunda Forma Normal (2FN): dependencia de toda la clave

La 2FN solo se puede incumplir en tablas con clave primaria compuesta: todo atributo no clave debe depender de la clave **completa**, no de una parte. En `tab_det_compra`, con clave (id_factura, id_prod), la cantidad y el precio unitario dependen de ambos campos, lo cual es correcto. Si se agregara `fec_factura` a esa tabla, dependería solo de id_factura (dependencia parcial) y se repetiría en cada línea; por eso vive en el encabezado. Lo mismo ocurriría con `nom_prod` en `tab_prod_prov`: depende solo de id_prod y su lugar es tab_prod. El esquema original ya cumplía la 2FN; conviene saber verificarlo.

Tabla	Clave primaria	Atributo	¿Depende de toda la clave?
tab_prod_prov	(id_prov, id_prod)	val_prod	Sí: el precio es de ese producto con ese proveedor
tab_det_compra	(id_factura, id_prod)	val_cant, val_unit	Sí: cantidad y precio de esa línea
tab_det_orden	(id_orden, id_prod)	val_cant	Sí
tab_pagos	(id_factura, id_pago)	fec_pago, val_pago	Sí: datos de esa cuota de esa factura
(hipotético) tab_det_compra	(id_factura, id_prod)	fec_factura	**No**: solo depende de id_factura → viola 2FN

Tercera Forma Normal (3FN): sin dependencias transitivas

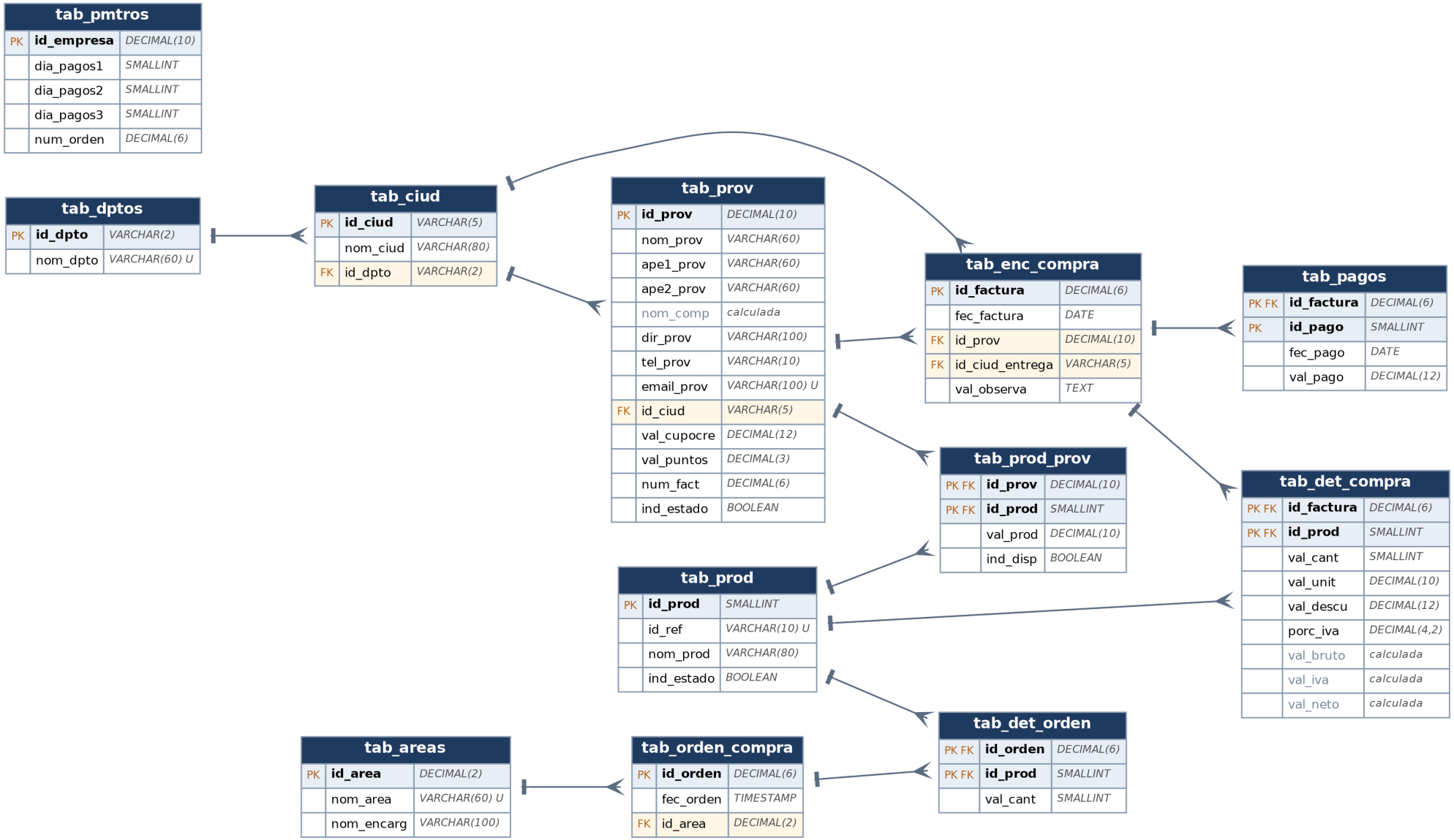
En 3FN ningún atributo no clave depende de otro atributo no clave. Los casos encontrados son todos datos **derivados**: valores que se pueden calcular a partir de otros. Guardarlos por separado obliga a mantenerlos sincronizados a mano, y el diagnóstico demuestra que no se logró: netos mal sumados, IVA inconsistente y saldos en cero con dinero pendiente.

Columna original	Se deriva de	Solución
tab_prov.nom_comp	nom_prov, ape1_prov, ape2_prov	Columna GENERATED: el motor la calcula
tab_det_compra.val_bruto	val_cant × val_unit	Columna GENERATED
tab_det_compra.val_iva	(bruto − descuento) × porc_iva	Columna GENERATED
tab_det_compra.val_neto	bruto − descuento + IVA	Columna GENERATED
tab_pagos.val_saldo	Σ val_neto − Σ val_pago	Se elimina; se calcula en la vista vw_estado_factura
tab_enc_compra.id_ciud (si fuera la del proveedor)	id_prov → tab_prov.id_ciud	Se redefine como ciudad de entrega (dato propio de la factura)

¿Una columna calculada no rompe la 3FN? Técnicamente el valor se almacena, pero lo calcula y mantiene el motor: no se puede escribir a mano ni quedar desactualizado, que es justamente lo que la normalización busca evitar. Es una desnormalización *controlada* que facilita las consultas. El saldo, en cambio, depende de filas de **otras** tablas, algo que una columna calculada no puede expresar; por eso se resuelve con una vista.

Observe también lo que **no** se guarda: el proveedor no tiene columna de departamento, porque el departamento se obtiene a partir de la ciudad (id_prov → id_ciud → id_dpto). Guardarlo sería una dependencia transitiva.

5. Modelo relacional resultante



PK = clave primaria, FK = clave foránea, U = restricción UNIQUE. Las columnas en gris son calculadas por el motor. Cada línea va del lado «uno» (barra) al lado «muchos» (pata de gallo).

Resumen de tablas e integridad referencial

La acción ante el borrado del registro padre se decide por el significado del dato. **RESTRICT** protege información que tiene valor propio (un departamento con ciudades, un proveedor con facturas, una factura con pagos). **CASCADE** se usa cuando el hijo no tiene sentido sin el padre (las líneas de una orden o de una factura). **ON UPDATE CASCADE** en los códigos geográficos permite corregir un código y propagarlo.

Tabla	Clave primaria	Claves foráneas (acción al borrar el padre)
tab_pmtros	id_empresa	—
tab_dptos	id_dpto	—
tab_ciud	id_ciud	id_dpto → tab_dptos (RESTRICT, ON UPDATE CASCADE)
tab_prov	id_prov	id_ciud → tab_ciud (RESTRICT, ON UPDATE CASCADE)
tab_prod	id_prod	— (clave alterna única: id_ref)
tab_prod_prov	(id_prov, id_prod)	id_prov → tab_prov (RESTRICT); id_prod → tab_prod (RESTRICT)
tab_areas	id_area	—
tab_orden_compra	id_orden	id_area → tab_areas (RESTRICT)
tab_det_orden	(id_orden, id_prod)	id_orden → tab_orden_compra (**CASCADE**); id_prod → tab_prod (RESTRICT)
tab_enc_compra	id_factura	id_prov → tab_prov (RESTRICT); id_ciud_entrega → tab_ciud (RESTRICT)
tab_det_compra	(id_factura, id_prod)	id_factura → tab_enc_compra (**CASCADE**); id_prod → tab_prod (RESTRICT)
tab_pagos	(id_factura, id_pago)	id_factura → tab_enc_compra (RESTRICT)

6. Creación de la base de datos (DDL)

El script de creación usa cinco herramientas de validación que conviene distinguir. **NOT NULL** obliga a que el dato exista. **DEFAULT** asigna un valor cuando no se informa. **CHECK** valida el dominio de una fila: rangos, formatos con expresiones regulares (operador ~, o ~* sin distinguir mayúsculas) y relaciones entre columnas de la misma fila. **UNIQUE** impide repetidos en claves alternas. **FOREIGN KEY** garantiza que el valor exista en otra tabla. Cuando la regla necesita consultar otras tablas, ninguna de ellas basta y se recurre a un **trigger**.

> Todas las restricciones llevan nombre (pk_, fk_, ck_, uq_). Así, cuando una validación falla, el mensaje de error dice exactamente qué regla se violó, como se verá en la sección 8.

6.1 Limpieza y parámetros

El DROP con CASCADE permite volver a ejecutar el script tantas veces como se quiera. En los parámetros, una sola restricción compara tres columnas de la misma fila para que los días de pago estén entre 1 (lunes) y 7 (domingo) y en orden creciente.

```
-- 0. Limpieza (CASCADE elimina también vistas y triggers dependientes)
DROP TABLE IF EXISTS tab_pagos, tab_det_compra, tab_enc_compra,
                     tab_det_orden, tab_orden_compra, tab_prod_prov,
                     tab_prov, tab_areas, tab_prod, tab_ciud,
                     tab_dptos, tab_pmtros CASCADE;
DROP FUNCTION IF EXISTS fn_valida_det_compra() CASCADE;
DROP FUNCTION IF EXISTS fn_valida_pago() CASCADE;

-- 1. Parámetros generales de la empresa
CREATE TABLE tab_pmtros
(
    id_empresa  DECIMAL(10,0) NOT NULL,
    dia_pagos1  SMALLINT      NOT NULL,   -- día de la semana: 1=lunes ... 7=domingo
    dia_pagos2  SMALLINT      NOT NULL,
    dia_pagos3  SMALLINT      NOT NULL,
    num_orden   DECIMAL(6,0)  NOT NULL,   -- consecutivo de órdenes de compra
    CONSTRAINT pk_pmtros       PRIMARY KEY (id_empresa),
    CONSTRAINT ck_pmtros_id    CHECK (id_empresa > 0),
    CONSTRAINT ck_pmtros_dias  CHECK (dia_pagos1 BETWEEN 1 AND 7
                                  AND dia_pagos2 BETWEEN 1 AND 7
                                  AND dia_pagos3 BETWEEN 1 AND 7
                                  AND dia_pagos1 < dia_pagos2
                                  AND dia_pagos2 < dia_pagos3),
    CONSTRAINT ck_pmtros_orden CHECK (num_orden > 0)
);
```

6.2 Geografía: departamentos y ciudades

`ck_ciud_prefijo` es una validación cruzada: el código de la ciudad debe empezar por el código de su departamento. `uq_ciud_nombre` evita registrar dos veces la misma ciudad en un departamento, pero permite nombres repetidos en departamentos distintos (Riosucio existe en Caldas y en Chocó).

```
-- 2. Departamentos
CREATE TABLE tab_dptos
(
    id_dpto   VARCHAR(2)  NOT NULL,
    nom_dpto  VARCHAR(60) NOT NULL,
    CONSTRAINT pk_dptos        PRIMARY KEY (id_dpto),
    CONSTRAINT uq_dptos_nombre UNIQUE (nom_dpto),
    CONSTRAINT ck_dptos_id     CHECK (id_dpto ~ '^[0-9]{2}$'),
    CONSTRAINT ck_dptos_nombre CHECK (BTRIM(nom_dpto) <> '')
);

-- 3. Ciudades (el código de la ciudad empieza por el código del dpto.)
CREATE TABLE tab_ciud
(
```

```
    id_ciud   VARCHAR(5)  NOT NULL,
    nom_ciud  VARCHAR(80) NOT NULL,
    id_dpto   VARCHAR(2)  NOT NULL,
    CONSTRAINT pk_ciud         PRIMARY KEY (id_ciud),
    CONSTRAINT fk_ciud_dpto    FOREIGN KEY (id_dpto)
               REFERENCES tab_dptos (id_dpto)
               ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_ciud_nombre  UNIQUE (id_dpto, nom_ciud),
    CONSTRAINT ck_ciud_id      CHECK (id_ciud ~ '^[0-9]{5}$'),
    CONSTRAINT ck_ciud_prefijo CHECK (LEFT(id_ciud, 2) = id_dpto),
    CONSTRAINT ck_ciud_nombre  CHECK (BTRIM(nom_ciud) <> '')
);
```

6.3 Proveedores

Aquí se aplican las correcciones de 1FN (contacto atómico) y 3FN (nombre completo calculado). El patrón del teléfono acepta fijos con indicativo 601, 602 o 604 a 608 y celulares que empiezan por 3, siempre con 10 dígitos.

```
-- 4. Proveedores (datos de contacto atómicos: cumple 1FN)
CREATE TABLE tab_prov
(
    id_prov      DECIMAL(10,0) NOT NULL,
    nom_prov     VARCHAR(60)   NOT NULL,
    ape1_prov    VARCHAR(60)   NOT NULL,
    ape2_prov    VARCHAR(60),
    nom_comp     VARCHAR(185)  GENERATED ALWAYS AS
                 (nom_prov || ' ' || ape1_prov || COALESCE(' ' || ape2_prov, ''))
                 STORED,
    dir_prov     VARCHAR(100)  NOT NULL,
    tel_prov     VARCHAR(10)   NOT NULL,
    email_prov   VARCHAR(100)  NOT NULL,
    id_ciud      VARCHAR(5)    NOT NULL,
    val_cupocre  DECIMAL(12,0) NOT NULL DEFAULT 0,
    val_puntos   DECIMAL(3,0)  NOT NULL DEFAULT 0,
    num_fact     DECIMAL(6,0)  NOT NULL,
    ind_estado   BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_prov         PRIMARY KEY (id_prov),
    CONSTRAINT fk_prov_ciud    FOREIGN KEY (id_ciud)
               REFERENCES tab_ciud (id_ciud)
               ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_prov_email   UNIQUE (email_prov),
    CONSTRAINT ck_prov_id      CHECK (id_prov > 0),
    CONSTRAINT ck_prov_tel     CHECK (tel_prov ~ '^(60[124-8]|3[0-9]{2})[0-9]{7}$'),
    CONSTRAINT ck_prov_email   CHECK (email_prov ~* '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$'),
    CONSTRAINT ck_prov_cupo    CHECK (val_cupocre >= 0),
    CONSTRAINT ck_prov_puntos  CHECK (val_puntos BETWEEN 0 AND 100),
    CONSTRAINT ck_prov_fact    CHECK (num_fact > 0)
);
```

6.4 Productos y catálogo por proveedor

`tab_prod_prov` resuelve la relación muchos a muchos entre proveedores y productos, y además guarda un atributo propio de la relación: el precio.

```
-- 5. Productos
CREATE TABLE tab_prod
(
    id_prod     SMALLINT    NOT NULL,
    id_ref      VARCHAR(10) NOT NULL,
    nom_prod    VARCHAR(80) NOT NULL,
    ind_estado  BOOLEAN     NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_prod        PRIMARY KEY (id_prod),
    CONSTRAINT uq_prod_ref    UNIQUE (id_ref),
    CONSTRAINT ck_prod_id     CHECK (id_prod > 0),
    CONSTRAINT ck_prod_ref    CHECK (id_ref ~ '^PROD[0-9]{3}$'),
    CONSTRAINT ck_prod_nombre CHECK (BTRIM(nom_prod) <> '')
);

-- 6. Productos que vende cada proveedor (relación N:M con atributos)
```

```
CREATE TABLE tab_prod_prov
(
    id_prov   DECIMAL(10,0) NOT NULL,
    id_prod   SMALLINT      NOT NULL,
    val_prod  DECIMAL(10,0) NOT NULL,
    ind_disp  BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_prod_prov      PRIMARY KEY (id_prov, id_prod),
    CONSTRAINT fk_prodprov_prov  FOREIGN KEY (id_prov)
               REFERENCES tab_prov (id_prov) ON DELETE RESTRICT,
    CONSTRAINT fk_prodprov_prod  FOREIGN KEY (id_prod)
               REFERENCES tab_prod (id_prod) ON DELETE RESTRICT,
    CONSTRAINT ck_prodprov_valor CHECK (val_prod > 0)
);
```

6.5 Áreas y órdenes de compra (encabezado–detalle)

```
-- 7. Áreas de la empresa
CREATE TABLE tab_areas
(
    id_area     DECIMAL(2,0)  NOT NULL,
    nom_area    VARCHAR(60)   NOT NULL,
    nom_encarg  VARCHAR(100)  NOT NULL,
    CONSTRAINT pk_areas        PRIMARY KEY (id_area),
    CONSTRAINT uq_areas_nombre UNIQUE (nom_area),
    CONSTRAINT ck_areas_id     CHECK (id_area > 0)
);

-- 8. Órdenes de compra: encabezado + detalle (una orden, varios productos)
CREATE TABLE tab_orden_compra
(
    id_orden   DECIMAL(6,0) NOT NULL,
    fec_orden  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_area    DECIMAL(2,0) NOT NULL,
    CONSTRAINT pk_orden      PRIMARY KEY (id_orden),
    CONSTRAINT fk_orden_area FOREIGN KEY (id_area)
               REFERENCES tab_areas (id_area) ON DELETE RESTRICT,
    CONSTRAINT ck_orden_id   CHECK (id_orden > 0)
);

CREATE TABLE tab_det_orden
(
    id_orden  DECIMAL(6,0) NOT NULL,
    id_prod   SMALLINT     NOT NULL,
    val_cant  SMALLINT     NOT NULL,
    CONSTRAINT pk_det_orden      PRIMARY KEY (id_orden, id_prod),
    CONSTRAINT fk_detorden_orden FOREIGN KEY (id_orden)
               REFERENCES tab_orden_compra (id_orden) ON DELETE CASCADE,
    CONSTRAINT fk_detorden_prod  FOREIGN KEY (id_prod)
               REFERENCES tab_prod (id_prod) ON DELETE RESTRICT,
    CONSTRAINT ck_detorden_cant  CHECK (val_cant > 0)
);
```

6.6 Facturas y pagos

El detalle almacena solo los datos base (cantidad, precio unitario, descuento y porcentaje de IVA) y deja que el motor calcule bruto, IVA y neto. Una columna calculada no puede referirse a otra calculada, por eso la expresión del neto repite el cálculo del IVA. La tabla de pagos ya no tiene saldo.

```
-- 9. Facturas de compra: encabezado
CREATE TABLE tab_enc_compra
(
    id_factura       DECIMAL(6,0)  NOT NULL,
    fec_factura      DATE          NOT NULL DEFAULT CURRENT_DATE,
    id_prov          DECIMAL(10,0) NOT NULL,
    id_ciud_entrega  VARCHAR(5)    NOT NULL,   -- ciudad donde se recibe la compra
    val_observa      TEXT,
    CONSTRAINT pk_enc_compra    PRIMARY KEY (id_factura),
    CONSTRAINT fk_enc_prov      FOREIGN KEY (id_prov)
               REFERENCES tab_prov (id_prov) ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_enc_ciud      FOREIGN KEY (id_ciud_entrega)
               REFERENCES tab_ciud (id_ciud) ON DELETE RESTRICT,
    CONSTRAINT ck_enc_id        CHECK (id_factura > 0),
    CONSTRAINT ck_enc_fecha     CHECK (fec_factura >= DATE '2000-01-01')
);

-- 10. Facturas de compra: detalle (bruto, IVA y neto se CALCULAN)
CREATE TABLE tab_det_compra
(
    id_factura  DECIMAL(6,0)  NOT NULL,
    id_prod     SMALLINT      NOT NULL,
    val_cant    SMALLINT      NOT NULL,
    val_unit    DECIMAL(10,0) NOT NULL,
    val_descu   DECIMAL(12,0) NOT NULL DEFAULT 0,
    porc_iva    DECIMAL(4,2)  NOT NULL DEFAULT 19,
    val_bruto   DECIMAL(12,0) GENERATED ALWAYS AS (val_cant * val_unit) STORED,
    val_iva     DECIMAL(12,0) GENERATED ALWAYS AS
                (ROUND((val_cant * val_unit - val_descu) * porc_iva / 100)) STORED,
    val_neto    DECIMAL(12,0) GENERATED ALWAYS AS
                (val_cant * val_unit - val_descu
                 + ROUND((val_cant * val_unit - val_descu) * porc_iva / 100)) STORED,
    CONSTRAINT pk_det_compra     PRIMARY KEY (id_factura, id_prod),
    CONSTRAINT fk_detcomp_enc    FOREIGN KEY (id_factura)
               REFERENCES tab_enc_compra (id_factura) ON DELETE CASCADE,
    CONSTRAINT fk_detcomp_prod   FOREIGN KEY (id_prod)
               REFERENCES tab_prod (id_prod) ON DELETE RESTRICT,
    CONSTRAINT ck_detcomp_cant   CHECK (val_cant > 0),
    CONSTRAINT ck_detcomp_unit   CHECK (val_unit > 0),
    CONSTRAINT ck_detcomp_descu  CHECK (val_descu >= 0
                                   AND val_descu <= val_cant * val_unit),
    CONSTRAINT ck_detcomp_iva    CHECK (porc_iva IN (0, 5, 19))
);

-- 11. Pagos (el saldo NO se almacena: se calcula en la vista)
CREATE TABLE tab_pagos
(
    id_factura  DECIMAL(6,0)  NOT NULL,
    id_pago     SMALLINT      NOT NULL,
    fec_pago    DATE          NOT NULL,
    val_pago    DECIMAL(12,0) NOT NULL,
    CONSTRAINT pk_pagos        PRIMARY KEY (id_factura, id_pago),
    CONSTRAINT fk_pagos_enc    FOREIGN KEY (id_factura)
               REFERENCES tab_enc_compra (id_factura) ON DELETE RESTRICT,
    CONSTRAINT ck_pagos_id     CHECK (id_pago > 0),
    CONSTRAINT ck_pagos_valor  CHECK (val_pago > 0)
);

-- Índices de apoyo para las claves foráneas más consultadas
CREATE INDEX idx_ciud_dpto   ON tab_ciud (id_dpto);
CREATE INDEX idx_prov_ciud   ON tab_prov (id_ciud);
CREATE INDEX idx_enc_prov    ON tab_enc_compra (id_prov);
CREATE INDEX idx_enc_fecha   ON tab_enc_compra (fec_factura);
CREATE INDEX idx_detcomp_prod ON tab_det_compra (id_prod);
```

7. Inserción de datos

Las claves foráneas determinan el orden de carga: primero las tablas que no dependen de nadie y al final las que dependen de varias. Si se intenta insertar una ciudad antes que su departamento, la clave foránea rechaza la fila. El orden de borrado es exactamente el inverso.

Paso	Tabla	Requiere que existan antes	Filas
1	tab_pmtros	—	1
2	tab_dptos	—	20
3	tab_ciud	tab_dptos	156
4	tab_areas	—	8
5	tab_prod	—	20
6	tab_prov	tab_ciud	20
7	tab_prod_prov	tab_prov, tab_prod	100
8	tab_orden_compra / tab_det_orden	tab_areas / tab_prod	30 / 30
9	tab_enc_compra	tab_prov, tab_ciud	60
10	tab_det_compra	tab_enc_compra, tab_prod_prov (trigger)	37
11	tab_pagos	tab_det_compra (trigger)	23

Buenas prácticas aplicadas

Toda la carga se hace dentro de una **transacción** (`BEGIN … COMMIT`): si una sola fila falla, no queda nada a medias. Cada INSERT lista sus columnas explícitamente, lo que hace el script independiente del orden físico de la tabla. Las columnas con DEFAULT (ind_estado, ind_disp, val_descu, porc_iva) pueden omitirse, y las columnas calculadas (nom_comp, val_bruto, val_iva, val_neto) **deben** omitirse. A continuación se muestran fragmentos representativos; el script completo está en el Anexo A.

Departamentos, ciudades y proveedores

```
BEGIN;

-- 2. Departamentos
INSERT INTO tab_dptos (id_dpto, nom_dpto) VALUES
('01', 'Amazonas'), ('02', 'Antioquia'), ('03', 'Arauca'), ('04', 'Atlántico'),
...

-- 3. Ciudades (Guaviare queda con sus 4 municipios reales)
INSERT INTO tab_ciud (id_ciud, nom_ciud, id_dpto) VALUES
('01001', 'Leticia', '01'), ('01002', 'Puerto Nariño', '01'), ('01003', 'Tarapacá', '01'),
('01004', 'Puerto Arica', '01'), ('01005', 'Puerto Santander', '01'), ('01006', 'La Pedrera', '01'),
...

-- 6. Proveedores (contacto en columnas atómicas; nom_comp se calcula solo)
INSERT INTO tab_prov (id_prov, nom_prov, ape1_prov, ape2_prov, dir_prov, tel_prov,
                      email_prov, id_ciud, val_cupocre, val_puntos, num_fact) VALUES
(1000000001, 'Carlos', 'Gómez', 'Hernández', 'Calle 100 #45-20', '6011234567',
 'carlos@tecnosum.com', '14001', 5000000, 95, 1000),
...
```

Catálogo, órdenes y facturas

```
-- 7. Productos por proveedor (precio de lista)
INSERT INTO tab_prod_prov (id_prov, id_prod, val_prod) VALUES
(1000000001, 1, 2500000), (1000000001, 2, 85000), (1000000001, 3, 120000), (1000000001, 4, 800000),
...

-- 8. Órdenes de compra (encabezado con fecha y HORA) y su detalle
```

```
INSERT INTO tab_orden_compra (id_orden, fec_orden, id_area) VALUES
(100001, '2024-01-15 08:30', 2),
(100002, '2024-01-16 09:15', 3),
...

-- 10. Facturas: detalle (solo datos base; bruto, IVA y neto son calculados)
INSERT INTO tab_det_compra (id_factura, id_prod, val_cant, val_unit, val_descu) VALUES
(200001, 1, 2, 2500000, 250000),
(200001, 2, 10, 85000, 42500),
(200002, 1, 3, 2500000, 375000),
...
```

Pagos y confirmación

Los pagos se recalcularon para que cuadren con los nuevos totales: las facturas 200001 a 200006 quedan pagadas por completo y las facturas 200007 a 200010 con pago parcial, lo que permite practicar consultas de cartera.

```
-- 11. Pagos (sin columna de saldo: el saldo se consulta en vw_estado_factura)
--     Facturas 200001-200006 quedan PAGADAS; 200007-200010 con PAGO PARCIAL
INSERT INTO tab_pagos (id_factura, id_pago, fec_pago, val_pago) VALUES
(200001, 1, '2024-02-05', 2000000),
(200001, 2, '2024-02-15', 2000000),
(200001, 3, '2024-02-25', 2613425),
(200002, 1, '2024-03-05', 4000000),
(200002, 2, '2024-03-15', 4000000),
(200002, 3, '2024-03-25', 1157050),
...
COMMIT;
```

Comprobación rápida después de la carga

```
SELECT estado, COUNT(*) AS facturas
  FROM vw_estado_factura
 GROUP BY estado;
```

estado	facturas
SIN DETALLE	34
PENDIENTE	16
PAGO PARCIAL	4
PAGADA	6

4 fila(s) en total.

8. Pruebas de validación e integridad

Una restricción que nunca se ha probado es solo una suposición. En esta sección se intenta, a propósito, guardar datos incorrectos. Cada instrucción **debe fallar**; el recuadro rojo muestra el mensaje real que devolvió PostgreSQL 16 (en una instalación en español el texto puede aparecer traducido, pero el nombre de la restricción es el mismo). Ejecute las pruebas una a una: como fallan, no modifican la base de datos.

Prueba 1. Integridad referencial: ciudad con un departamento inexistente

```
INSERT INTO tab_ciud (id_ciud, nom_ciud, id_dpto)
VALUES ('99001', 'Ciudad Fantasma', '99');
```

```
ERROR: insert or update on table "tab_ciud" violates foreign key constraint "fk_ciud_dpto"
```

La clave foránea fk_ciud_dpto impide crear una ciudad «huérfana».

Prueba 2. Validación de formato: el código de la ciudad no coincide con el departamento

```
INSERT INTO tab_ciud (id_ciud, nom_ciud, id_dpto)
VALUES ('02009', 'Sabaneta', '05');
```

```
ERROR: new row for relation "tab_ciud" violates check constraint "ck_ciud_prefijo"
```

La restricción ck_ciud_prefijo exige que los dos primeros dígitos sean el código del departamento.

Prueba 3. Validación de dominio: correo electrónico mal formado

```
INSERT INTO tab_prov (id_prov, nom_prov, ape1_prov, dir_prov, tel_prov,
                      email_prov, id_ciud, num_fact)
VALUES (1000000099, 'Juan', 'Pérez', 'Calle 1 #2-3', '6011112233',
        'juan.perez@correo', '14001', 21000);
```

```
ERROR: new row for relation "tab_prov" violates check constraint "ck_prov_email"
```

La expresión regular de ck_prov_email exige usuario@dominio.extensión.

Prueba 4. Validación de rango: puntaje de proveedor fuera de 0-100

```
UPDATE tab_prov SET val_puntos = 150
 WHERE id_prov = 1000000001;
```

```
ERROR: new row for relation "tab_prov" violates check constraint "ck_prov_puntos"
```

ck_prov_puntos limita el puntaje al intervalo [0, 100].

Prueba 5. Validación de formato: teléfono con prefijo inexistente

```
UPDATE tab_prov SET tel_prov = '6031234567'
 WHERE id_prov = 1000000009;
```

```
ERROR: new row for relation "tab_prov" violates check constraint "ck_prov_tel"
```

En Colombia los fijos usan los indicativos 601, 602 y 604 a 608 (el 603 no existe) y los móviles empiezan por 3.

Prueba 6. Unicidad: referencia de producto repetida

```
INSERT INTO tab_prod (id_prod, id_ref, nom_prod)
VALUES (21, 'PROD001', 'Laptop Oficina');
```

```
ERROR: duplicate key value violates unique constraint "uq_prod_ref"
```

uq_prod_ref garantiza que la clave alterna id_ref no se repita.

Prueba 7. Clave primaria: área duplicada

```
INSERT INTO tab_areas (id_area, nom_area, nom_encarg)
VALUES (1, 'Jurídica', 'Paula Ríos');
```

```
ERROR: duplicate key value violates unique constraint "pk_areas"
```

No pueden existir dos filas con el mismo id_area.

Prueba 8. Regla de negocio (trigger): facturar un producto que el proveedor no vende

```
INSERT INTO tab_det_compra (id_factura, id_prod, val_cant, val_unit)
VALUES (200001, 20, 1, 400000);
```

```
ERROR: El producto 20 no lo vende (o no está disponible) el proveedor de la factura 200001
```

El proveedor 1000000001 (factura 200001) solo vende los productos 1 a 5.

Prueba 9. Columna calculada: intentar escribir el valor neto

```
INSERT INTO tab_det_compra (id_factura, id_prod, val_cant, val_unit, val_neto)
VALUES (200011, 14, 1, 120000, 999);
```

```
ERROR: cannot insert a non-DEFAULT value into column "val_neto"
```

val_neto es GENERATED ALWAYS: el motor la calcula y no admite valores manuales.

Prueba 10. Validación de cantidad: descuento mayor que el valor bruto

```
INSERT INTO tab_det_compra (id_factura, id_prod, val_cant, val_unit, val_descu)
VALUES (200011, 14, 1, 120000, 500000);
```

```
ERROR: new row for relation "tab_det_compra" violates check constraint "ck_detcomp_descu"
```

ck_detcomp_descu impide un descuento superior al bruto (neto negativo).

Prueba 11. Regla de negocio (trigger): pago anterior a la fecha de la factura

```
INSERT INTO tab_pagos (id_factura, id_pago, fec_pago, val_pago)
VALUES (200011, 1, '2024-01-01', 100000);
```

```
ERROR: Fecha de pago 2024-01-01 anterior a la factura (2024-03-01)
```

La factura 200011 es del 2024-03-01; no se puede pagar antes de existir.

Prueba 12. Regla de negocio (trigger): pago superior al saldo

```
INSERT INTO tab_pagos (id_factura, id_pago, fec_pago, val_pago)
VALUES (200007, 2, '2024-04-20', 9000000);
```

```
ERROR: El pago (9000000) supera el saldo de la factura 200007 (saldo: 2991650)
```

La factura 200007 tiene un saldo pendiente inferior a 9.000.000.

Prueba 13. Integridad referencial (RESTRICT): borrar un departamento con ciudades

```
DELETE FROM tab_dptos WHERE id_dpto = '02';
```

```
ERROR: update or delete on table "tab_dptos" violates foreign key constraint "fk_ciud_dpto" on table
"tab_ciud"
```

ON DELETE RESTRICT protege a las 8 ciudades de Antioquia.

Prueba 14. Integridad referencial (RESTRICT): borrar una factura que ya tiene pagos

```
DELETE FROM tab_enc_compra WHERE id_factura = 200001;
```

```
ERROR: update or delete on table "tab_enc_compra" violates foreign key constraint "fk_pagos_enc" on
table "tab_pagos"
```

Los pagos son evidencia contable: fk_pagos_enc no permite eliminarlos en cascada.

Prueba 15. Validación de parámetros: días de pago desordenados

```
UPDATE tab_pmtros SET dia_pagos1 = 6 WHERE id_empresa = 1;
```

```
ERROR: new row for relation "tab_pmtros" violates check constraint "ck_pmtros_dias"
```

ck_pmtros_dias exige días entre 1 y 7 y en orden creciente.

Prueba 16. Borrado en cascada (resultado esperado: éxito)

La factura 200019 tiene una línea de detalle y ningún pago. Al borrar su encabezado, ON DELETE CASCADE elimina también el detalle. La prueba se hace dentro de una transacción con ROLLBACK para no perder los datos.

```
BEGIN;
SELECT COUNT(*) AS lineas_antes FROM tab_det_compra WHERE id_factura = 200019;
DELETE FROM tab_enc_compra WHERE id_factura = 200019;   -- sin pagos asociados
SELECT COUNT(*) AS lineas_despues FROM tab_det_compra WHERE id_factura = 200019;
ROLLBACK;   -- deshacemos la prueba
```

```
lineas_antes = 1 → lineas_despues = 0 (la línea se borró junto con su factura)
```

9. Consultas

Las consultas están organizadas en tres niveles. Antes de leer la solución, intente escribir cada consulta a partir del enunciado. Todos los resultados mostrados son reales y se obtuvieron sobre los datos de la sección 7; los valores monetarios están en pesos colombianos con punto como separador de miles.

Nivel 1 · Consultas básicas

Consulta 1. Catálogo de productos activos [SELECT, WHERE, ORDER BY]

Listar la referencia y el nombre de los productos activos, ordenados por referencia.

```
SELECT id_ref, nom_prod
  FROM tab_prod
 WHERE ind_estado = TRUE
 ORDER BY id_ref;
```

id_ref	nom_prod
PROD001	Laptop Gamer
PROD002	Mouse Inalámbrico
PROD003	Teclado Mecánico
PROD004	Monitor 24"
PROD005	Impresora Láser
PROD006	Tablet 10"
PROD007	Smartphone
PROD008	Auriculares Bluetooth

20 fila(s) en total; se muestran las primeras 8.

Consulta 2. Ciudades de un departamento [Filtro por clave foránea]

Mostrar las ciudades registradas para Antioquia (código 02).

```
SELECT id_ciud, nom_ciud
  FROM tab_ciud
 WHERE id_dpto = '02'
 ORDER BY nom_ciud;
```

id_ciud	nom_ciud
02002	Bello
02007	Copacabana
02004	Envigado
02003	Itagüí
02008	La Estrella
02006	Marinilla
02001	Medellín
02005	Rionegro

8 fila(s) en total.

Consulta 3. Proveedores destacados [Operadores lógicos AND]

Proveedores con cupo de crédito de al menos 5.000.000 y puntaje mínimo de 90.

```
SELECT nom_comp, val_cupocre, val_puntos
  FROM tab_prov
 WHERE val_cupocre >= 5000000
   AND val_puntos  >= 90
 ORDER BY val_cupocre DESC;
```

nom_comp	val_cupocre	val_puntos
Luis Rodríguez Pérez	8.000.000	92
Sofía Ramírez González	6.000.000	94
Roberto Navarro Suárez	5.800.000	96
David Torres Vargas	5.500.000	91
Andrés Ortega Navarro	5.200.000	92
Carlos Gómez Hernández	5.000.000	95

6 fila(s) en total.

Consulta 4. Búsqueda por patrón de texto [ILIKE y comodines]

Productos cuyo nombre contiene «USB» o «inalámbrico», sin distinguir mayúsculas.

```
SELECT id_ref, nom_prod
  FROM tab_prod
 WHERE nom_prod ILIKE '%usb%'
    OR nom_prod ILIKE '%inalámbrico%';
```

id_ref	nom_prod
PROD002	Mouse Inalámbrico
PROD014	Micrófono USB

2 fila(s) en total.

Nivel 2 · Combinación de tablas y agregación

Consulta 5. Proveedores con su ubicación [INNER JOIN de tres tablas]

Nombre completo del proveedor con su ciudad y departamento.

```
SELECT p.nom_comp, c.nom_ciud, d.nom_dpto
  FROM tab_prov  p
  JOIN tab_ciud  c ON c.id_ciud = p.id_ciud
  JOIN tab_dptos d ON d.id_dpto = c.id_dpto
 ORDER BY d.nom_dpto, p.nom_comp;
```

nom_comp	nom_ciud	nom_dpto
Ana Martínez López	Medellín	Antioquia
Carmen Vargas Mendoza	Envigado	Antioquia
Gabriela Rojas Pérez	Saravena	Arauca
Lucía Fernández Romero	Tame	Arauca
Pedro Hernández Silva	Arauca	Arauca
María López García	Barranquilla	Atlántico
Patricia Mendoza Rojas	Cartagena	Bolívar
Andrés Ortega Navarro	Manizales	Caldas

20 fila(s) en total; se muestran las primeras 8.

Consulta 6. Departamentos con pocas ciudades [GROUP BY y HAVING]

Departamentos con menos de 8 ciudades registradas.

```
SELECT d.nom_dpto, COUNT(c.id_ciud) AS ciudades
  FROM tab_dptos d
  LEFT JOIN tab_ciud c ON c.id_dpto = d.id_dpto
 GROUP BY d.nom_dpto
HAVING COUNT(c.id_ciud) < 8;
```

nom_dpto	ciudades
Guaviare	4

1 fila(s) en total.

Consulta 7. Rango de precios por producto [Funciones de agregación]

Para cada producto: número de proveedores, precio mínimo, máximo y promedio.

```
SELECT pr.nom_prod,
       COUNT(*)                 AS proveedores,
       MIN(pp.val_prod)         AS precio_min,
       MAX(pp.val_prod)         AS precio_max,
       ROUND(AVG(pp.val_prod))  AS precio_prom
  FROM tab_prod_prov pp
  JOIN tab_prod pr ON pr.id_prod = pp.id_prod
 GROUP BY pr.nom_prod
 ORDER BY precio_prom DESC;
```

nom_prod	proveedores	precio_min	precio_max	precio_prom
Tarjeta Gráfica	4	3.050.000	3.200.000	3.125.000
Laptop Gamer	5	2.300.000	2.500.000	2.400.000
Smartphone	6	1.650.000	1.800.000	1.716.667
Proyector	4	1.400.000	1.500.000	1.442.500
Tablet 10"	5	1.050.000	1.200.000	1.116.000
Monitor 24"	5	690.000	800.000	736.000
Escritorio Oficina	4	575.000	600.000	586.250
Silla Ergonómica	5	420.000	450.000	433.000

20 fila(s) en total; se muestran las primeras 8.

Consulta 8. Proveedor más económico por producto [Subconsulta correlacionada]

Para cada producto, el proveedor que ofrece el menor precio.

```
SELECT pr.nom_prod, p.nom_comp, pp.val_prod
  FROM tab_prod_prov pp
  JOIN tab_prod pr ON pr.id_prod = pp.id_prod
  JOIN tab_prov p  ON p.id_prov  = pp.id_prov
 WHERE pp.val_prod = (SELECT MIN(x.val_prod)
                        FROM tab_prod_prov x
                       WHERE x.id_prod = pp.id_prod)
 ORDER BY pr.nom_prod;
```

nom_prod	nom_comp	val_prod
Altavoces	Miguel Suárez Romero	155.000
Auriculares Bluetooth	Lucía Fernández Romero	135.000
Cámara Web	Roberto Navarro Suárez	78.000
Disco Duro SSD 1TB	Lucía Fernández Romero	315.000
Escritorio Oficina	Roberto Navarro Suárez	575.000
Fuente de Poder	Isabel Pérez Morales	168.000
Impresora Láser	Miguel Suárez Romero	395.000
Laptop Gamer	Elena Morales Fernández	2.300.000

20 fila(s) en total; se muestran las primeras 8.

Consulta 9. Total comprado por proveedor [LEFT JOIN, SUM y COALESCE]

Número de facturas y valor neto comprado a cada proveedor (incluye los que aún no tienen detalle).

```
SELECT p.nom_comp,
       COUNT(DISTINCT e.id_factura)  AS facturas,
       COALESCE(SUM(d.val_neto), 0)  AS total_neto
  FROM tab_prov p
  LEFT JOIN tab_enc_compra e ON e.id_prov    = p.id_prov
  LEFT JOIN tab_det_compra d ON d.id_factura = e.id_factura
 GROUP BY p.nom_comp
 ORDER BY total_neto DESC;
```

nom_comp	facturas	total_neto
Ana Martínez López	3	25.775.400
Carlos Gómez Hernández	3	20.405.525
Luis Rodríguez Pérez	3	17.296.650
Sofía Ramírez González	3	15.448.283
María López García	3	11.067.595
Carmen Vargas Mendoza	3	8.297.870
David Torres Vargas	3	7.755.230
Pedro Hernández Silva	3	6.285.580

20 fila(s) en total; se muestran las primeras 8.

Consulta 10. Facturas sin detalle [NOT EXISTS (anti-join)]

Encabezados de factura que no tienen ninguna línea de detalle (información incompleta).

```
SELECT e.id_factura, e.fec_factura, e.id_prov
  FROM tab_enc_compra e
 WHERE NOT EXISTS (SELECT 1
                     FROM tab_det_compra d
                    WHERE d.id_factura = e.id_factura)
 ORDER BY e.id_factura;
```

id_factura	fec_factura	id_prov
200021	2024-04-10	1000000007
200024	2024-04-15	1000000008
200027	2024-04-20	1000000009
200030	2024-04-25	1000000010
200031	2024-03-10	1000000011
200032	2024-04-05	1000000011

34 fila(s) en total; se muestran las primeras 6.

Estas 34 facturas son los encabezados sin detalle del diagnóstico (hallazgo 15). Completar su detalle es uno de los ejercicios de la sección 10.

Consulta 11. Unidades solicitadas por área [JOIN + GROUP BY sobre órdenes]

Cantidad de órdenes y unidades pedidas por cada área.

```
SELECT a.nom_area,
       COUNT(DISTINCT o.id_orden) AS ordenes,
       SUM(d.val_cant)            AS unidades
  FROM tab_areas a
  JOIN tab_orden_compra o ON o.id_area  = a.id_area
  JOIN tab_det_orden    d ON d.id_orden = o.id_orden
 GROUP BY a.nom_area
 ORDER BY unidades DESC;
```

nom_area	ordenes	unidades
Ventas	8	148
Compras	8	117
Operaciones	7	114
Tecnología	7	81

4 fila(s) en total.

Nivel 3 · Consultas avanzadas

Consulta 12. Estado de cuenta de facturas [Uso de una VISTA]

Facturas con saldo pendiente, calculado a partir de la vista vw_estado_factura.

```
SELECT id_factura, proveedor, val_total, val_pagado, val_saldo, estado
  FROM vw_estado_factura
 WHERE val_saldo > 0
 ORDER BY val_saldo DESC;
```

id_factura	proveedor	val_total	val_pagado	val_saldo	estado
200028	Carmen Vargas Mendoza	7.348.250	0	7.348.250	PENDIENTE
200016	Sofía Ramírez González	7.054.320	0	7.054.320	PENDIENTE
200017	Sofía Ramírez González	5.935.125	0	5.935.125	PENDIENTE
200025	David Torres Vargas	4.476.780	0	4.476.780	PENDIENTE
200019	Pedro Hernández Silva	3.843.700	0	3.843.700	PENDIENTE
200013	Jorge Díaz Ramírez	3.594.990	0	3.594.990	PENDIENTE
200026	David Torres Vargas	3.278.450	0	3.278.450	PENDIENTE
200007	Luis Rodríguez Pérez	5.991.650	3.000.000	2.991.650	PAGO PARCIAL

20 fila(s) en total; se muestran las primeras 8.

Observe que este saldo no está guardado en ninguna tabla: se calcula en el momento a partir del detalle y de los pagos.

Consulta 13. Cartera vencida a una fecha de corte [Aritmética de fechas y CASE]

Facturas con saldo y su antigüedad al 30 de junio de 2024, clasificadas por rango de días.

```
SELECT id_factura, fec_factura,
       DATE '2024-06-30' - fec_factura AS dias,
       val_saldo,
       CASE
         WHEN DATE '2024-06-30' - fec_factura > 120 THEN 'Más de 120 días'
         WHEN DATE '2024-06-30' - fec_factura >  90 THEN '91 a 120 días'
         ELSE '90 días o menos'
       END AS rango
  FROM vw_estado_factura
 WHERE val_saldo > 0
 ORDER BY dias DESC;
```

id_factura	fec_factura	dias	val_saldo	rango
200007	2024-01-30	152	2.991.650	Más de 120 días
200010	2024-02-05	146	715.170	Más de 120 días
200013	2024-02-10	141	3.594.990	Más de 120 días
200016	2024-02-15	136	7.054.320	Más de 120 días
200019	2024-02-20	131	3.843.700	Más de 120 días
200022	2024-02-25	126	1.582.700	Más de 120 días
200008	2024-02-25	126	1.270.100	Más de 120 días
200025	2024-03-01	121	4.476.780	Más de 120 días

20 fila(s) en total; se muestran las primeras 8.

Consulta 14. Compras mensuales y acumulado [DATE_TRUNC y SUM() OVER]

Valor neto comprado por mes y su acumulado en el año.

```
SELECT TO_CHAR(DATE_TRUNC('month', e.fec_factura), 'YYYY-MM') AS mes,
       SUM(d.val_neto)                                        AS compras_mes,
       SUM(SUM(d.val_neto)) OVER (ORDER BY DATE_TRUNC('month', e.fec_factura))
                                                              AS acumulado
  FROM tab_enc_compra e
  JOIN tab_det_compra d ON d.id_factura = e.id_factura
 GROUP BY DATE_TRUNC('month', e.fec_factura)
 ORDER BY 1;
```

mes	compras_mes	acumulado
2024-01	18.031.475	18.031.475
2024-02	52.070.830	70.102.305
2024-03	49.476.333	119.578.638
2024-04	2.458.838	122.037.476

4 fila(s) en total.

Consulta 15. Ranking de productos más comprados [Función de ventana RANK()]

Productos ordenados por valor neto comprado, con su posición.

```
SELECT RANK() OVER (ORDER BY SUM(d.val_neto) DESC) AS puesto,
       p.nom_prod,
       SUM(d.val_cant) AS unidades,
       SUM(d.val_neto) AS total_neto
  FROM tab_det_compra d
  JOIN tab_prod p ON p.id_prod = d.id_prod
 GROUP BY p.nom_prod
 ORDER BY puesto
 LIMIT 5;
```

puesto	nom_prod	unidades	total_neto
1	Smartphone	11	21.988.225
2	Tablet 10"	12	16.053.100
3	Memoria RAM 16GB	48	14.538.230
4	Laptop Gamer	5	14.131.250
5	Monitor 24"	10	8.704.850

5 fila(s) en total.

Consulta 16. Ahorro potencial frente al mejor precio [CTE (WITH)]

Compara el precio unitario pagado con el mejor precio disponible en el mercado.

```
WITH mejor_precio AS (
     SELECT id_prod, MIN(val_prod) AS precio_min
       FROM tab_prod_prov
      GROUP BY id_prod
)
SELECT d.id_factura, p.nom_prod, d.val_cant,
       d.val_unit, m.precio_min,
       (d.val_unit - m.precio_min) * d.val_cant AS sobrecosto
  FROM tab_det_compra d
  JOIN mejor_precio m ON m.id_prod = d.id_prod
  JOIN tab_prod     p ON p.id_prod = d.id_prod
 WHERE d.val_unit > m.precio_min
 ORDER BY sobrecosto DESC
 LIMIT 6;
```

id_factura	nom_prod	val_cant	val_unit	precio_min	sobrecosto
200005	Tablet 10"	5	1.200.000	1.050.000	750.000
200006	Smartphone	4	1.800.000	1.650.000	600.000
200002	Laptop Gamer	3	2.500.000	2.300.000	600.000
200004	Tablet 10"	3	1.200.000	1.050.000	450.000
200007	Memoria RAM 16GB	15	280.000	250.000	450.000
200003	Monitor 24"	4	800.000	690.000	440.000

6 fila(s) en total.

Consulta 17. Auditoría: precios facturados vs. lista [Consulta de control de calidad]

Líneas de factura cuyo precio unitario difiere del precio de lista del proveedor. Un resultado vacío confirma la consistencia.

```
SELECT d.id_factura, d.id_prod, d.val_unit, pp.val_prod
  FROM tab_det_compra d
  JOIN tab_enc_compra e  ON e.id_factura = d.id_factura
  JOIN tab_prod_prov  pp ON pp.id_prov = e.id_prov
                        AND pp.id_prod = d.id_prod
 WHERE d.val_unit <> pp.val_prod;
```

id_factura	id_prod	val_unit	val_prod

(0 filas: no hay inconsistencias)

0 fila(s) en total.

10. Ejercicios propuestos

Los ejercicios siguen el mismo orden de la guía: diseño, validación, carga y consulta. Varios de ellos producen errores intencionales; anote el mensaje y explique qué regla lo produjo.

N.º	Tema	Enunciado
1	Diseño y normalización	Las órdenes de compra y las facturas no están relacionadas. Proponga cómo registrar qué orden originó cada factura. ¿Basta una columna id_orden en tab_enc_compra o se necesita una tabla intermedia? Justifique según la cardinalidad (una factura puede atender varias órdenes y una orden puede facturarse por partes).
2	Normalización	Se quiere guardar varios teléfonos por proveedor (fijo, celular, WhatsApp). Explique por qué agregar columnas tel2 y tel3 no es una buena solución y diseñe la tabla tab_tel_prov con su clave primaria, clave foránea y validaciones.
3	Validación	Agregue a tab_prov una restricción que impida que un proveedor inactivo (ind_estado = FALSE) tenga cupo de crédito mayor que cero. Pruébela con un UPDATE válido y otro inválido.
4	Integridad referencial	Intente borrar el producto 1 (Laptop Gamer). ¿Qué restricción lo impide primero? En lugar de borrarlo, ¿qué cambio de datos sería el adecuado para retirarlo del catálogo?
5	Triggers	Desactive la disponibilidad del producto 4 para el proveedor 1000000010 (ind_disp = FALSE) e intente agregarlo a la factura 200029. Explique el resultado y restaure el valor.
6	Inserción de datos	Complete el detalle de tres de las facturas sin detalle (Consulta 10), usando solo productos del catálogo de su proveedor y el precio de lista. Verifique luego su estado en vw_estado_factura.
7	Inserción de datos	Registre el pago que salda por completo la factura 200007. Calcule primero el saldo con la vista y luego intente pagar 1 peso más de lo debido para comprobar el trigger.
8	Consultas	Liste las áreas que no han generado ninguna orden de compra. Resuélvalo de dos formas: con LEFT JOIN … IS NULL y con NOT EXISTS.
9	Consultas	Para cada departamento, muestre cuántos proveedores tiene y el cupo de crédito total, incluyendo los departamentos sin proveedores (con cero).
10	Consultas	Usando ROW_NUMBER() o DISTINCT ON, reescriba la Consulta 8 (proveedor más económico por producto) y compare los resultados. ¿Qué pasaría si dos proveedores tuvieran el mismo precio mínimo?
11	Mejora del modelo	Reemplace el consecutivo num_orden de tab_pmtros por una SEQUENCE de PostgreSQL y úsela como valor por defecto de id_orden. Discuta qué ventajas tiene ante usuarios concurrentes.
12	Análisis	La vista calcula el saldo cada vez. Si la empresa tuviera millones de pagos, ¿qué alternativas existen (vista materializada, columna mantenida por trigger)? ¿Qué riesgo de la sección 3 vuelve a aparecer en cada caso?

Pistas de autoevaluación

En el ejercicio 4, la primera restricción que actúa depende del orden interno en que PostgreSQL revisa las claves foráneas; lo importante es identificar que tab_prod_prov, tab_det_orden y tab_det_compra lo referencian. La forma correcta de retirar un producto es marcarlo inactivo, lo que además lo bloquea para nuevas facturas gracias al trigger. En el ejercicio 8 las áreas sin órdenes son Administración, Recursos Humanos, Calidad y Logística. En el ejercicio 7 el saldo de la factura 200007 es 2.991.650.